

Assignment 1 – Report

For ease of writing, reading, and grading, I have structured this report according to the headings in the instructions. I am aware that the report is very long (sorry!), but so was the assignment and I want my report to reflect the many, many hours I spent working on it. All code and technical reproduction steps are available on [GitHub](#).

The data

Computation or storage was not an issue for me in this assignment (at least not due to data size), so I did not see any need or use for subsampling of the data.

The tokenizers

Using the combined provided training data, I built and trained a character-level tokenizer from scratch and trained one small and one big BPE tokenizer via the Hugging Face Tokenizers library. I will refer to these and their associated models as “Char-level,” “Small BPE,” and “Big BPE” from now on. I added two special tokens to all three tokenizers: an [UNK] unknown token to handle unseen tokens, and an [EOS] end-of-sentence token to separate sentences during model training. Other special tokens ([BOS], [PAD], [MASK]) were considered but ultimately excluded since I did not find a need for them.

For the BPE tokenizers, I also elected to use Metaspace pre-tokenization, which basically means that whitespace in regular text is replaced by an underscore (`_`). This was done for two reasons. First, I wanted to avoid merging across word limits, since that makes tokens less linguistically meaningful. When I first built my BPE tokenizers I therefore used the Whitespace pre-tokenization recommended in [Tokenizers’ quick-tour guide](#). However, Whitespace pre-tokenization makes sample tokenizations more difficult to read, since words are then no longer separated. Metaspace was therefore a good compromise: I could avoid merging across word limits while still keeping word separation in the output. As will be seen in “Who gets the vocabulary?”, Metaspace also helped me notice an interesting tokenization distribution pattern.

Diverging from the recommended BPE vocabulary sizes of 2,000 and 10,000 tokens, I instead chose to use 10,000 and 20,000. The reason for this was that the initial alphabet – i.e. all characters in the training data + my two special tokens – alone amounted to 9,444 tokens. Using only 2,000 tokens for Small BPE would therefore not only cut much of the initial alphabet but also remove the possibility of adding any merged tokens. Especially the latter problem would, in my opinion, defeat the purpose of even using a BPE tokenizer in the first place, so I decided to increase the vocabulary sizes a little ($9,444 + 556 = 10,000$) and a lot ($9,444 + 10,556 = 20,000$) beyond the initial alphabet. This way, I could compare how adding few vs. many merged tokens affected the tokenization.

Before you run the experiment

Below are the predictions I wrote down before running the experiment. I have cleaned up the language somewhat compared to the original formulations (available on GitHub as `pre-experiment_analysis.txt`), but the content is the same.

1. **Token usage:** I expect Chinese to require the most tokens, partly since a Chinese character contains so much meaning compared to an English and (I assume) an Turkish character, and partly because Chinese (therefore) needs a much larger alphabet.
2. **Impact of larger BPE vocabulary on language representation:** For English and possibly Turkish, I expect a larger BPE vocabulary to include more short words and sub-words. For Chinese, however, I expect few merges even for a big BPE vocabulary (for the same reasons as in #1).
3. **Token characteristics:** For English, I expect BPE to learn common short words (“the,” “and”) and sub-words (“ing,” “er”). For Turkish, which is the language with which I have the least experience, I do not know what to expect at all, though given its Latin alphabet I guess BPE

learning will be somewhat similar to English. For Chinese, as mentioned, I expect very few merges if any, and if so only really by Big BPE.

4. **Tokenizer performance on language modelling:** For English and possibly Turkish, I expect Big BPE to perform the best for language modelling, since I know BPE was the preferred option for the original breakthrough LLMs. For Chinese, I predict all tokenizers to be approximately equally good (or bad), since I know from before that tokenization has been a common pain point for Chinese.

Analyze the tokenizers

Below are the **vocabulary sizes** of the different tokenizers. They stay the same regardless of language and dataset. Please see the reasoning under “The tokenizers” for why I chose these BPE numbers.

- **Char-level:** 9,444 tokens
- **Small BPE:** 10,000 tokens
- **Big BPE:** 20,000 tokens

Below are the **average Unicode characters per token** for the different tokenizers. These statistics also stay the same regardless of language and dataset. The reason why the value is >1 for Char-level is that its vocabulary also includes two special tokens ([UNK] and [EOS]), each of which is five Unicode characters long. The small gap between Char-level and Small BPE makes sense because Small BPE had very little room to add multi-character merges that would make a difference for this statistic (556 vocabulary spots). Big BPE, having much more such space (10,556 vocabulary spots), was able to increase the average considerably.

- **Char-level:** 1.00085 chars/token
- **Small BPE:** 1.1036 chars/token
- **Big BPE:** 2.52545 chars/token

Below are the **total tokens needed to encode the validation data** for the different tokenizers and languages. The character counts for the three datasets are roughly the same (about 360,000 characters), which substantiates the idea that downstream language comparisons will be fair.

	en	tr	zh
Char-level	368,269	368,383	368,266
Small BPE	174,617	182,138	346,050
Big BPE	107,828	106,077	267,425

That fact that Char-level > Small BPE > Big BPE overall is expected since especially Big BPE but also Small BPE have access to more space-efficient multi-character tokens. This inequality is very noticeable for English and Turkish, with Small BPE requiring roughly half and Big BPE roughly a third of the number of tokens used by Char-level. For Chinese, however, the differences are much more subtle – Small BPE is roughly comparable to Char-level, and the gap between Char-level and Big BPE is almost half as small as even the gap between Char-level and Small BPE in the other languages. This suggests that multilingual BPE allocates merged tokens to English and Turkish more often than to Chinese.

Below are the **average tokens per validation-data sentence** for the different tokenizers and languages. The patterns here are broadly similar to the previous statistic, with Char-level > Small BPE > Big BPE across the board and the gaps being much smaller for Chinese.

	en	tr	zh
Char-level	81.1	98.6	38.5
Small BPE	38.5	49.0	36.2
Big BPE	23.8	28.4	28.0

However, one interesting difference is that Char-level average token/sentence is much lower for Chinese than for the other languages. This suggests that while the three datasets contain roughly the same number of characters overall, the Chinese sentences are much shorter on average. In turn, this means that Chinese tokenization generally uses the special separator token more often (if available). Another consequence is that average tokens/sentence for BPE tokenization is comparable for all three languages: the benefit of more multi-character tokens in English and Turkish is counterbalanced by their generally longer sentences.

Below are **tokenizations of one sample sentence per language** by the different tokenizers (more are available in `tokenizer_analysis.txt` on GitHub). I will discuss each language in turn. For more information about the underscores in the BPE tokenizations, please see “The tokenizers.”

Sample sentence (EN): In 1536, the Act of Union was passed under Henry's rule which had a long-lasting effect on Wales as a nation.

- **Char-level:** In 1536, the Act of Union was passed under Henry's rule which had a long-lasting effect on Wales as a nation.
- **Small BPE:** _In _1536, _the _A ct _of _U nion _was _p as sed _un der _H en ry's _r ule _which _had _a _l ong -l as ting _e ffect _on _W ales _as _a _n ation .
- **Big BPE:** _In _1536, _the _Act _of _Union _was _pass ed _under _Henry's _rule _which _had _a _long -l as ting _effect _on _W ales _as _a _n ation .

In English, Small BPE has picked up on not only many short sub-words (“un”, “ed”) and words (“was”, “to”), but – surprisingly given its small room for merges – even relatively long patterns consisting of five characters (“which”, “ation”). This suggests that even early BPE merges are being used generously for English. AS expected, Big BPE then picks up on most common words and sub-words, with splits mostly remaining for compound words (“long - l as ting”), names (“W ales”), and punctuation (“H en ry’s”).

Sample sentence (TR): Germantown akademisine devam ettikten sonra 1869-1870 yılları Fransa ve Almanya'da okullarda geçti.

- **Char-level:** Germantown akademisine devam ettikten sonra 1869-1870 yılları Fransa ve Almanya'da okullarda geçti.
- **Small BPE:** _G er man t own _a k ad em is ine _dev am _et tik ten _son ra _1869-1870 _yıl ları _Fr an sa _ve _Al man ya 'da _o k ul lar da _g eç ti .
- **Big BPE:** _German t own _ak adem is ine _devam _et tik ten _son ra _1869-1870 _yılları _Fransa _ve _Alman ya 'da _okul lar da _g eç ti .

Turkish, which sported similar training data statistics to English, also demonstrates similar tokenization patterns. Small BPE seems to be able to pick up on short sub-words and words (“ye”, “ine”) and even relatively long patterns (“sonra”). Big BPE still splits up punctuation (“g eç ti .”) and names (“German t own”), though more common names like “Fransa” and “Almanya” actually do get dedicated tokens. In fact, it is somewhat surprising that the English-sounding “Germantown” stays so disjointed despite the tokenizer being trained partly on English.

Sample sentence (ZH): 藝術家會在作品中控制灰分的百分比，若降低灰的比例，控制的程度更高 Ford & Impey, 46-50。

- **Char-level:** 藝術家會在作品中控制灰分的百分比，若降低灰的比例，控制的程度更高 Ford & Impey, 46-50。
- **Small BPE:** _藝 術 家 會 在 作 品 中 控 制 灰 分 的 百 分 比 ， 若 降 低 灰 的 比 例 ， 控 制 的 程 度 更 高 _F ord _& _I m pey , _46-50 _。

- **Big BPE:** 藝術家會在作品中控制灰分的百分比，若降低灰的比例，控制的程度更高。Ford & Imp ey, 46 - 50。

In Chinese, Small BPE barely changes the tokenization compared to Char-level, and the few merges are in fact not specifically Chinese (“ord”, “ey”, “50”). One could easily mistake Small BPE for being an English- and/or Turkish-only tokenizer. In contrast, Big BPE does in fact merge some Chinese characters (“藝術”, “降低”), but rarely more than two at a time. As mentioned, this is likely a consequence of most of the merges being used for English and Turkish.

Overall, I am surprised that Small BPE is so effective for English and Turkish despite having so little room for merged tokens. This suggests to me that the first merges are the ones that matter the most, which would make sense since those are also the most common multi-character combinations. While I have not explicitly investigated it for tokenization, this seems to be Zipf’s law in action.

The language models

I trained one decoder-only Transformer for each tokenization condition using these hyperparameters:

- **Transformer layers:** 2 (recommended)
- **Hidden dimensions:** 256 (recommended)
- **Attention heads:** 4 (recommended)
- **Feed-forward dimensions:** 1024 (recommended)
- **Dropout rate:** 0.1
 - While dropout in general was recommended, the specific dropout rate was not. I chose 0.1 as a somewhat arbitrary first choice and then kept it since training seemed to go well. Testing different values would admittedly be more rigorous, however.
- **Context length:** 256 (recommended)
- **Epochs:** 8
 - As can be seen on the validation curves under “Training” below, my models started seeing diminishing returns already around 3-4 epochs, but even at 8 epochs the validation loss was still decreasing, albeit slowly. I used 5 epochs for my checkpoint and considered trying 10 epochs, but due to the training time and the limited time available I decided that 8 was good enough.
- **Batch size:** 32
 - I tried 64 at first but the GPU then ran out of memory, so I had to lower it.
- **Learning rate:** 0.001
 - Again, this was a somewhat arbitrary first choice that seemed to perform well and was therefore kept.

Fair comparisons

I chose to keep the epochs and therefore the amount of underlying text constant, as changing training hyperparameters would – in my opinion – make the experiment less realistic and therefore less useful. In real life, I imagine you would likely try different plug-and-play tokenizers with default settings, select the one that performs the best as-is on your downstream task, and only then start adjusting hyperparameters. Nevertheless, this is admittedly an argument based on practicality more so than fairness. In terms of fairness, my policy is justified partly because it keeps model training consistent across tokenization conditions, and partly because it exposes models to the same amount of raw linguistic data.

That said, my approach is not perfect. Keeping epochs and the amount of underlying text constant means that a model coupled with a character-level tokenizer would have more prediction targets to learn from than a BPE-based model, since literally every character then is a separate learning opportunity. Similarly, training data in English and Turkish would provide fewer prediction targets for

BPE-based models than otherwise equal training data in Chinese, as the former can be represented using fewer tokens. In that sense, my policy could be criticized as unfair, as different tokenization approaches result in different numbers of prediction targets and optimizer updates. Even then, however, smaller but more plentiful tokens do not obviously constitute *better* training than larger but fewer tokens: the training is just different.

Model size

Using PyTorch's built-in parameters method, I get these **total parameter counts** for my three models:

- **Char-level:** 6,490,340 parameters
- **Small BPE:** 6,775,568 parameters
- **Big BPE:** 11,905,568 parameters

Using the given formula for the number of parameters only in the **input embedding matrix** (hidden dimensions x vocabulary size), I get these parameter counts:

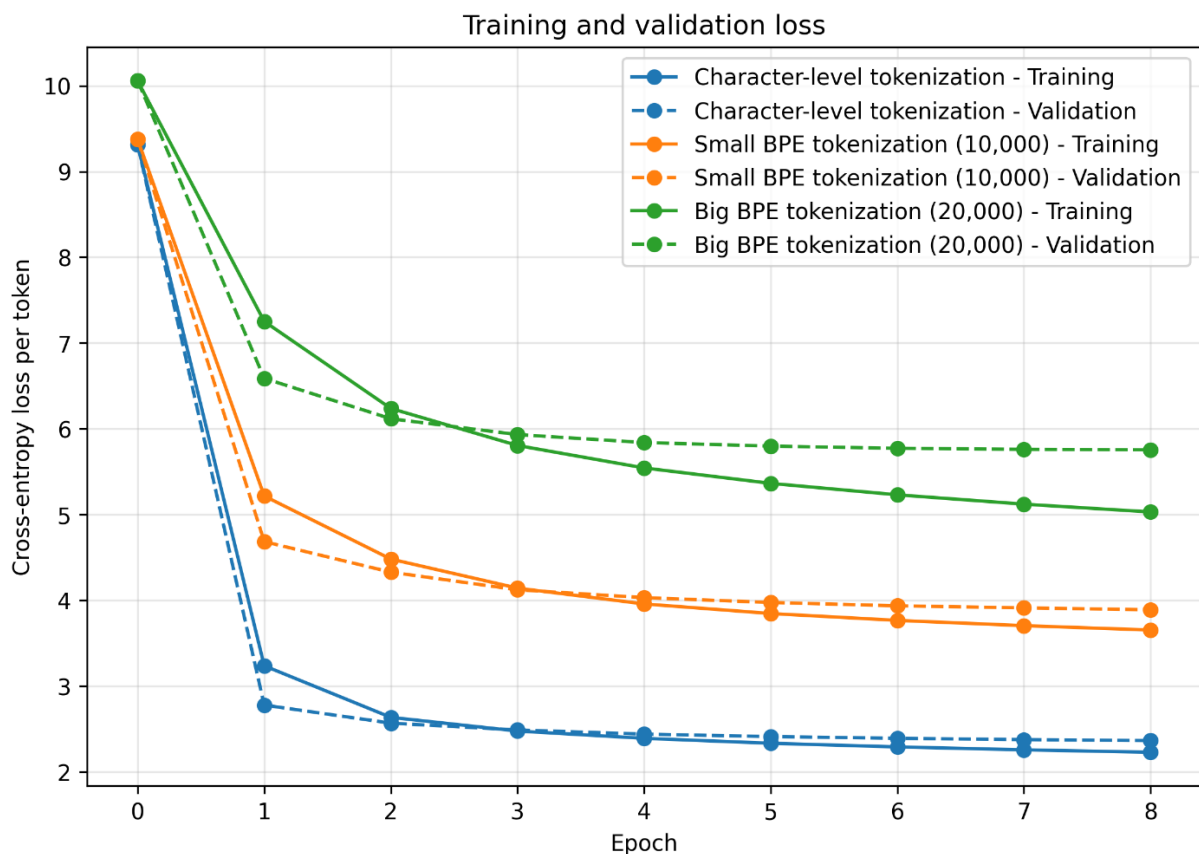
- **Char-level:** $256 \times 9,444 = 2,417,664$ parameters
- **Small BPE:** $256 \times 10,000 = 2,560,000$ parameters
- **Big BPE:** $256 \times 20,000 = 5,120,000$ parameters

The only difference in the **output vocabulary layer** is then that there is also a bias term providing an extra vocabulary size's worth of parameters:

- **Char-level:** $(256 + 1) \times 9,444 = 2,427,108$ parameters
- **Small BPE:** $(256 + 1) \times 10,000 = 2,570,000$ parameters
- **Big BPE:** $(256 + 1) \times 20,000 = 5,140,000$ parameters

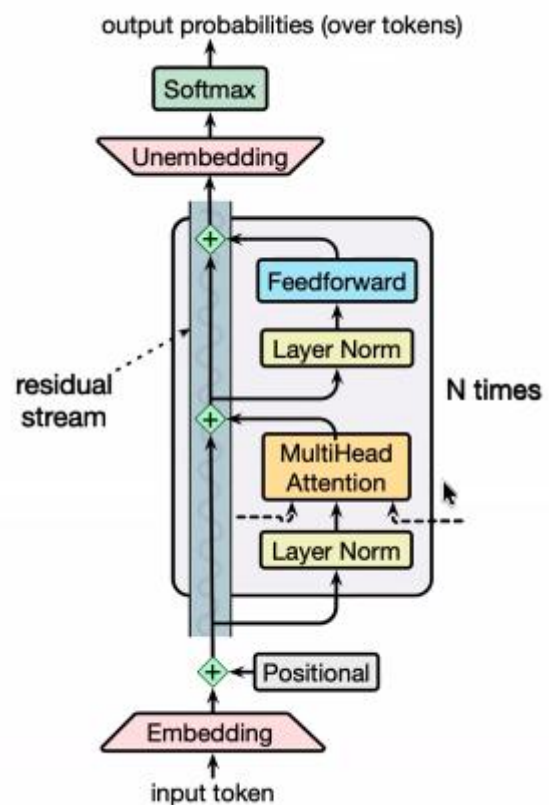
Training

Below is a plot of the training and validation loss across epochs for the different models.



My model architecture consists of token and positional embeddings, two Transformer blocks, a final normalization layer (since I use pre-normalization in the Transformer blocks), and an unembedding/output layer. The Transformer blocks themselves each contained a multi-head attention block and a feedforward neural network with GELU non-linearity coupled with linear layers on either end and final dropout. The multi-head attention and feedforward blocks were each preceded by a normalization layer and succeeded by a merging with a residual stream. The architecture therefore largely mirrors the architecture in the figure to the right (taken from Sharid's presentation), except there is an extra normalization layer after the Transformer blocks and no softmax, since I am just comparing losses.

I trained the models for 8 epochs with an Adam optimizer (learning rate = 0.001) and cross-entropy loss. Further discussion about the training-control policy and hyperparameters can be found under "Fair comparisons" and "The language models." The trained models are available on GitHub in the folder titled "models."



I encountered more problems and made more failed choices than I have space to explain here, so I will highlight just a few I can remember on top of my head:

- The recommended small BPE vocabulary size of 2,000 confused me a lot at first until I read up on the difference between alphabet and vocabulary and discussed it with Maria.
- I did not realize that PyTorch's CrossEntropyLoss expected the class variable as the second parameter, so that caused some errors before I finally reshaped the input matrices.
- The mlt-gpu GPU ran out of memory when I tried a batch size of 64, so I had to lower it to 32.
- At first, the methods and saved JSON format of my CharTokenizer class did not fully align with HuggingFace's Tokenizer class, causing problems for downstream functions that were supposed to operate on either type of object.
- I ran into an error when trying to load a model from a file since the default context window parameter (128) I had carelessly set did not align with the one I had actually used for the saved model (256).
- I struggled with how to apply causal attention masking until I found a guide on YouTube.
- As discussed above (under "The tokenizers"), I first used Whitespace pre-tokenization before switching to Metaspaces pre-tokenization.
- My character-level tokenizer did not include an [EOS] special token at first, which caused problems during model training.

Checkpoint before final submission

For this section, I refer to the separate document submitted earlier this week.

Evaluation and analysis

I evaluated each model on the English, Turkish, and Chinese test data both jointly and separately.

Comparing models

Since I used cross-entropy loss, which is measured in nats rather than bits, I selected nats per character (NPC) over bits per character (BPC) as my normalized measure. BPC is just NPC divided by a constant ($\ln(2)$) anyway, so the relative comparisons between languages and models should stay the same. Below is the **overall nats per character (NPC) on the entire test dataset** for each model:

- **Char-level:** 2.5513 NPC
- **Small BPE:** 2.5373 NPC
- **Big BPE:** 2.5952 NPC

Both NPC and BPC measure how surprised a language model is by a text after normalizing for character count, with lower values being better. Judging from the statistics above, models are evidently remarkably comparable, with Big BPE being ever so slightly worse than both Char-level and Small BPE. However, the gaps are so small that it is difficult to tell whether they actually represent true differences or just random variation. Either way, I find it very surprising that Big BPE does not perform noticeably better than Small BPE (or even Char-level), despite its larger vocabulary.

For more fine-grained analysis, I also compiled the **NPC for each model x language combination** on the given test data.

	en	tr	zh
Char-level	1.534	1.553	4.566
Small BPE	1.485	1.522	4.602
Big BPE	1.488	1.549	4.746

The Chinese test data is clearly more surprising to the models than that of the other two languages, regardless of tokenizer type. Since modelling Chinese is a considerably more difficult prediction problem (with more characters to choose from), this makes sense. Another interesting observation is that, even within each language, models stay remarkably comparable. This suggests that the findings above hold even at the language-specific level: in other words, tokenizers that are comparable in one language are likely also comparable in another.

I will now return to two pre-experiment hypotheses and one pre-experiment question.

1. **Chinese will require the most tokens.** This turned out to be true (for the BPE tokenizers, which are the ones of interest here): Chinese required about twice the number of tokens compared to English and Turkish (346k vs. 175k and 182k tokens for Small BPE and 267k vs. 108k and 106k tokens for Turkish). In my initial hypothesis I speculated that this difference was partly due to each Chinese character containing more meaning than each English and Turkish character, and partly due to the (therefore) vastly larger Chinese alphabet. I still believe this to be true.
2. **Big BPE will perform the best for English and Turkish.** This turned out to be false: A generous interpretation would be that Big BPE performed similarly to Small BPE, but my numbers suggest that it even performed slightly worse (1.488 vs. 1.485 NPC for English and 1.549 vs 1.522 NPC for Turkish). Upon further thought, this could be a result of several factors. First, given my training policy of keeping model parameters constant, Big BPE was able to represent the training data with fewer tokens and therefore had fewer opportunities to learn. Second, as suggested in the qualitative analysis of sample sentences (under “Analyze the tokenizers”), the first merges likely matter the most because they are also the most common. Third, this conversely means that the additional 10,000 merges in the Big BPE vocabulary possibly hurt

more than helped, since if they were not being predicted often, they were just diluting the pool of possible alternatives and thus making prediction more difficult.

3. **Is there ever a point in using character-level tokenization over BPE tokenization?** I initially doubted the virtues of character-level tokenization. However, given that character-level tokenization performs similarly to both small and big BPE on the test data (≈ 1.5 NPC for English and Turkish; ≈ 4.6 NPC for Chinese) while being significantly simpler to implement, I have now changed my mind: Character-level tokenization is in fact surprisingly useful – especially if you don’t want to spend too much time and effort on optimizing your tokenization. That said, these results have to be taken with a grain of salt, as Char-level had the smallest vocabulary (9,444 vs. 10,000 and 20,000 tokens), little vocabulary bloating compared to Big BPE (which included many low-frequency merged tokens), and more learning opportunities to learn, since it represented the training data using the most tokens (e.g. 368k vs. 175k and 108k tokens for English).

Who gets the vocabulary?

My criterion for assigning a token to a language is that at least 30% of the total occurrences of that token are in the training data of that particular language. This allows for two-way classifications and (very rarely) three-way classifications.

Below is the **token distribution for Small BPE**:

- **English:** 2.99%
- **Turkish:** 3.37%
- **Chinese:** 90.22%
- **Shared:** 3.43%

Below is the **token distribution for Big BPE**:

- **English:** 13.59%
- **Turkish:** 14.50%
- **Chinese:** 65.81%
- **Shared:** 6.10%

My checkpoint did not really prompt any questions related to token distribution. However, in my tokenizer analysis that was done at approximately the same time (tokenizer_analysis.txt on GitHub), I wondered “how much ‘help’ e.g. Turkish gets from English merges, and vice versa.” Specifically investigating this, I found that 9 Small BPE and 100 Big BPE tokens were shared between English and Turkish only. The most common Small BPE tokens of this kind (>100 occurrences) were “_her” and “_hav”, with Big BPE also adding “_dam”, “_top”, “_organ”, “_oper”, and “_final” to the list.

Interestingly, all of the most common shared tokens are prefixes, i.e. tokens occurring at the beginning of a word (and thus starting with a Metaspace underscore). In fact, the same seems to apply to the list of shared English-Turkish tokens as a whole, with 87/109 shared tokens occurring more than once in both languages being prefixes. This suggests to me that English and Turkish share many word beginnings, such as “_organiz” and “_polit”, but not so many word endings. The type of “help” each language can receive from the merges of the other is thus mostly concentrated to prefixes.

Digging deeper into the statistics, it turns out that 1.83% of Small BPE and 4.94% of Big BPE tokens are shared between English and Turkish only. While these numbers are small on their own, they add considerably to the uniquely English (2.99%, 13.59%) and Turkish vocabularies (3.37%, 14.50%), effectively expanding each by about 50%. In this sense, the amount of “help” each language gets from the other is actually quite sizeable.

Relating tokenization to model behavior

One pattern from my checkpoint training curves that I found especially interesting was the difference in how much the different models learned during training. Pre-training, the cross-entropy loss per token for all three models started at around 10 nats. After only one epoch, however, Char-level had reached about 3 nats, Small BPE about 5 nats, and big BPE about 7 nats, and these relative gaps between the models then stayed at approximately 2 nats throughout the rest of the training. I expected neither these gap sizes nor this gap consistency.

Even more confusing, the NPC statistics later showed that the three models in fact performed more or less equally on language prediction. The different amounts of learning thus did not seem to matter for the downstream task. How could that be?

One explanation might be that the models were solving different prediction problems. Char-level, with a vocabulary size of 9,444 and around 368k tokens per language, had both fewer alternatives to choose from during prediction and more opportunities to learn compared to the BPE models. Small BPE then increased the difficulty by expanding the vocabulary size slightly to 10,000 and cutting the total training token counts by almost half. Lastly, Big BPE doubled down by increasing vocabulary size and decreasing training tokens even further. Big BPE was therefore trained on a much more difficult task than Char-level, making it much clearer why its loss did not fall as much during training.

In the end, what this teaches me that I need to be more critical of loss comparisons, since they may in fact not represent learning for the same task.

Error analysis

I found it interesting how the token distribution changed when BPE vocabulary size increased: The proportion of uniquely Chinese tokens fell dramatically (90.22% → 65.81%), while the unique English and Turkish vocabularies more than quadrupled in relative size (2.99% → 13.59% and 3.37% → 14.50%). This suggests that increased BPE vocabulary capacity is disproportionately used to create English and Turkish tokens over Chinese. However, is this general trend noticeable also on a local level?

Below are tokenizations of one randomly picked sentence per language. The number of tokens for each sentence is 49, 30, and 70 when tokenized by Small BPE and 31, 17, and 51 when tokenized by Big BPE. The percentage decrease is thus 36.7% for English, 43.3% for Turkish, and 27.1% for Chinese. So, at least for these particular sentences, increased BPE vocabulary indeed resulted in more English and Turkish tokens than Chinese.

Sample sentence (EN): The corals will become bleached (lose their colours) and many species that live on and around the reef will be in danger.

- **Small BPE:** _The _c or al s _w ill _bec ome _b le ac he d _ (l os e _their _c ol our s) _and _m any _sp ec ies _that _l ive _on _and _ar ound _the _re e f _w ill _be _in _d ang er .
- **Big BPE:** _The _cor als _will _become _b le ached _ (l ose _their _colour s) _and _many _species _that _live _on _and _around _the _re ef _will _be _in _d anger .

Sample sentence (TR): Kayseri iline 54 km, Bünyan ilçesine 19 km uzaklıktadır.

- **Small BPE:** _K ay s eri _il ine _ 5 4 _k m , _B ün y an _il ç es ine _19 _k m _ uz ak lık tad ır .
- **Big BPE:** _K ays eri _iline _5 4 _km, _B ün yan _ilçesine _19 _km _uzaklık tad ır .

Sample sentence (ZH): 這不是外來的概念，傳統是由瑪泰在死亡前以一種被稱為「馬卡加」(mavaega)的意志形式出現，後來則由公共信託辦公室和法律從業者負責處理遺產管理。

- **Small BPE:** —這不是外來的概念，傳統是由瑪泰在死亡前以一種被稱為「馬卡加」(mavaega)的意志形式出現，後來則由公共信託辦公室和法律從業者負責處理遺產管理。
- **Big BPE:** —這不是外來的概念，傳統是由瑪泰在死亡前以一種被稱為「馬卡加」(mavaega)的意志形式出現，後來則由公共信託辦公室和法律從業者負責處理遺產管理。

Why is this, however? My interpretation is that, unlike languages like English and Turkish, much of the meaning in Chinese is already adequately captured by individual characters. Therefore, Chinese needs a big initial alphabet but not so many merges to be well represented by BPE, while for English and Turkish the relationship is the opposite. In this assignment, Small BPE and Big BPE also shared the same initial alphabet of 9,444 characters – a majority of which were Chinese. Much of the vocabulary needed to represent Chinese was therefore already present from the start, before any merges had taken place at all. This further reduced the need for additional Chinese tokens when the BPE vocabulary was expanded.

Conclusion

Returning to my pre-experiment expectations, one hypothesis that was supported by the experiment was that all models would perform equally well/badly on Chinese. They all hovered around 4.6 NPC – much higher than the 1.4-1.6 range associated with English and Turkish. On the other hand, an expectation that was not supported was that Big BPE would perform the best for English and Turkish. In fact, Big BPE performed at about the same level as Char-level and Small BPE, and – if my results are accurate – even slightly worse than Small BPE (1.488 vs. 1.485 NPC for English and 1.549 vs. 1.522 NPC for Turkish). This came as a big surprise to me.

All in all, I think this experiment has showed at least two things about using a multilingual tokenizer for English, Turkish, and Chinese. First, that character-level tokenizers are surprisingly competitive compared to BPE tokenizers: Char-level scored about the same NPC as both Small BPE and Big BPE (around 1.5 on English and Turkish and around 4.6 on Chinese). I should note that this may at least partly be a result of my particular training-control policy, however. Secondly, the experiment demonstrated that modelling Chinese is difficult for both character-level and BPE tokenizers: As mentioned, Chinese scored triple the NPC of the other languages regardless of tokenization approach.

Nevertheless, if I were forced to choose a tokenization approach for this particular context, I would probably select character-level tokenization. Why? It is relatively simple to implement, very interpretable, surprisingly competitive (see above), easily adaptable to other contexts, and – because it is so simple – it has fewer potential failure points (such as vocabulary size hyperparameters), which also makes it safer to use. Combined, these arguments have swayed a former BPE fan.